

```
Apr 16 17:27
oslab@oslab-2025S: ~/assgn2

[INFO 0,-1] bootcpu_entry: basic smp init, thread_id available now, we are cpu 0: 0x00000000802310f0
Kernel Starts Relocating...
Kernel size: 0x000000000032000, Rounded to 2MiB: 0x000000000200000
[INFO 0,-1] bootcpu_start_relocation: Kernel phy_base: 0x0000000080200000, phy_end_4k:0x0000000080232000, phy_end_2M 0x0000000080400000
Mapping Identity: 0x0000000080200000 to 0x0000000080200000
Mapping kernel image: 0xffffffff80200000 to 0x0000000080200000
Mapping Direct Mapping: 0xffffffffc080400000 to 0x0000000080400000
Enable SATP on temporary pagetable.
Boot HART Relocated. We are at high address now! PC: 0xffffffff80203df0
[INFO 0,-1] kvm_init: boot-stage page allocator: base 0xffffffffc080400000, end 0xffffffffc080600000
[INFO 0,-1] kvmmake: Memory after kernel image (phys) size = 0x0000000007c00000
[INFO 0,-1] kvm_init: enable pageing at 0x800000000080400
[INFO 0,-1] kvm_init: boot-stage page allocator ends up: base 0xffffffffc080400000, used: 0xffffffffc080411000
Relocated. Boot halt sp at 0xffffffffffff001fa0
Boot another cpus.
- booting hart 1: hsm_hart_start(hartid=1, pc=_entry_sec, opaque=1) = -3.
skipped for hart 1
- booting hart 2: hsm_hart_start(hartid=2, pc=_entry_sec, opaque=1) = -3.
skipped for hart 2
- booting hart 3: hsm_hart_start(hartid=3, pc=_entry_sec, opaque=1) = -3.
skipped for hart 3
System has 1 cpus online

UART initied.
[INFO 0,-1] kpgmgrinit: page allocator init: base: 0xffffffffc080411000, stop: 0xffffffffc088000000
[INFO 0,-1] allocator_init: allocator mm initied base 0xfffffffffd00000000
[INFO 0,-1] allocator_init: allocator vma initied base 0xfffffffffd01000000
[INFO 0,-1] allocator_init: allocator proc initied base 0xfffffffffd02000000
[INFO 0,-1] allocator_init: allocator kstrbuf initied base 0xfffffffffd03000000
applist:
    init
    proctest
    sh
    test_arg
    test_malloc
    test_uaccess
[INFO 0,-1] load_init_app: load init proc init
[INFO 0,-1] bootcpu_init: start scheduler!
init: starting sh
sh >> test_uaccess 1
-> checkpoint 1 passed
sh > child 3 exit with code 0
sh >> test_uaccess 2
test write
-> checkpoint 2 passed
sh > child 4 exit with code 0
sh >> test_uaccess 3
-> kernel secret addr: 0xFFFFFFFF8020E0D8
-> Your copy_from_user should not leak kernel data
-> checkpoint 3 passed
sh > child 5 exit with code 0
sh >> test_uaccess 4
-> checkpoint 4 passed
sh > child 6 exit with code 0
sh >>
```

完成作业耗时约3个小时